

3. Skills, Connectors, and Plugins

Rice Business 2026

Kerry Back
Rice Business

Three Ways to Extend Claude

Skills

- Teach Claude *your procedure* for a task
- It follows the same steps every time

Connectors

- Give Claude *your data and systems*
- Ask a database, your calendar, your files

Plugins

- *Bundle* skills and connectors into one package
- Install once; share a whole setup with a team

The model is fixed. What you change is what it *knows how to do* (skills), what it *can reach* (connectors), and how you *package and share* both (plugins).

Skills

Codified Expertise, Not a Smarter Model

The problem with chatting

- A great prompt works once, then scrolls out of your history
- The next person — or the next you — starts over
- Quality depends on remembering how you did it last time

What a skill fixes

- Write down “how to do this well,” once
- Every future run follows it — same standard, no reminding
- A **one-time workflow** becomes a **reusable asset**

A skill is not a model upgrade. It is your procedure, written down where Claude will follow it every time — across Claude.ai, Claude Code, and the add-ins.

What a Skill Actually Is

A folder and a file

- A SKILL.md file: a name, a description, and instructions in plain English
- Optional scripts and reference documents alongside
- That is the whole thing — readable, editable, versionable

How Claude uses it

- At startup it reads only each skill's name and description
- When a task looks relevant, it pulls in the full instructions
- Scripts run only when needed — so your context stays lean

This “progressive disclosure” is why you can have dozens of skills installed and pay attention-cost only for the one in use.

Three Ways a Skill Fires

Slash command

You type `/critique` or `/pptx` to invoke it directly

Natural language

You ask in plain English and Claude runs the matching skill

Automatic

“Create an Excel file” triggers the Excel skill on its own — Claude reads each skill’s description and picks

Because each skill carries a short description Claude always sees, it knows which ones exist and when to reach for them — without you naming them.

Build Your Own

From Workflow to Skill

The easy path

- Type `/skill-creator` and describe your workflow
- It writes the `SKILL.md` and saves it where Claude will find it
- Invoke it by name from then on

Bake in the checks

- Put your verification steps *inside* the skill
- “After computing, show the row count and the min and max”
- The skill enforces your standard, every run — not just when you remember

The best skills encode not just what to do, but how to *check* it — your discipline, made automatic.

Two Skills You Might Write

Teaching

Turn a problem set into a solution key, a grading rubric, and three fresh variants — formatted your way, every time

Research

Audit a replication package: run the code, check each table against the paper, report what does not reproduce

A hand-written skill — yours, transparent, under your control — is safer and more teachable than a multi-skill black box you did not read.

Connectors

Plugging Claude Into Your Systems

MCP: the universal plug

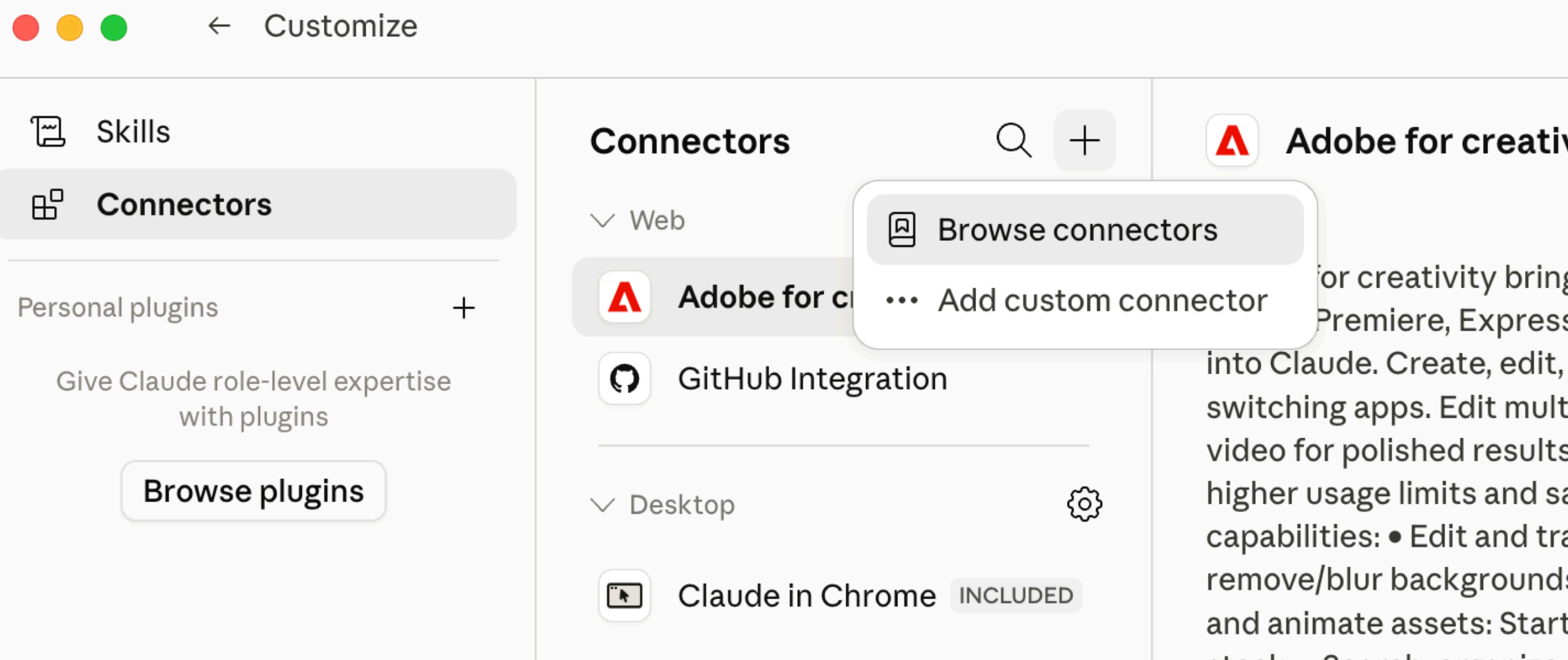
- A **connector** links Claude to an outside service — a database, your email, a document store
- The standard is MCP, the Model Context Protocol
- MCP is to AI what USB is to devices: one way to plug anything in

What it enables

- “Ask our enrollment database in plain English”
- “Check my calendar and draft the replies”
- Add from a directory of ready-made connectors, or point it at your own

Connectors are how Claude stops being a general assistant and starts knowing *your* data — safely and centrally, rather than through copy-paste.

Adding a Connector



A connector needs a name and an address; sensitive ones authenticate the way any app does — an API key, or a corporate single sign-on. Once added, its tools are available in every chat.

Plugins

Bundling It All to Share

What a plugin is

- A single installable package that can bundle several pieces at once
- The way a team distributes a shared setup — everyone installs one thing
- `/plugin install` and you have the whole kit

What it can contain

- Skills, custom agents, connectors (MCP servers)
- Hooks and output styles that shape behavior
- Skills are for *you*; plugins are for a *team*

For your own work, a skill is enough. When a department wants everyone working the same way, a plugin ships the whole standard in one command.

Vetting

Installing Someone Else's Runs Their Code

The supply-chain risk

- A shared skill, connector, or plugin runs someone else's instructions and scripts on your machine
- Many live in independent repos, outside Anthropic's screened directory
- There is active security research on malicious third-party skills

Due diligence

- `claude plugin validate` checks *structure*, not safety
- Read the `SKILL.md` and any scripts yourself
- Fine for a class demo; think twice before pointing it at real data

"Validate" confirms a package is well-formed, not that it is trustworthy. The safest skill is often the small one you wrote and can read in full.

The Payoff

Skills turn your best practice into something Claude does the same way every time. **Connectors** give it your data instead of a copy-paste. **Plugins** hand a colleague the whole setup in one command. You keep the judgment; they carry the discipline.

Breakout

Questions to Discuss

- What is one task — teaching or research — you do the same careful way each time, a good candidate for a skill?
- What system or dataset would you most want to *ask* in plain English — a good connector to build?
- Would you install a colleague's skill or plugin? What would you read before trusting it with your data?
- If your department adopted one shared plugin, what should be in it?